Taller práctico · Bloque 7

Del modelo de clases orientado a objetos a un diseño por tres capas para una API REST

Caso de estudio SmartLibrary — caso de uso «Registrar préstamo»

Dato	Valor
Asignatura	Diseño de Software — Cuarto semestre
Integrante 1	_____
Integrante 2	_____
Profesor	_____
Fecha	_____

Contenido y entregables

Entregable solicitado	Dónde está
Diagrama inicial anotado con observaciones	Actividad 1 (Figura 1)
Tabla de responsabilidades completa	Actividad 2
Diagrama de clases por capas con relaciones y multiplicidades justificadas	Actividad 3 (Figura 2 y tabla de relaciones)
Diagrama de secuencia del préstamo	Actividad 4 (Figura 3)
Proyecto Spring Boot y evidencias de pruebas	Actividades 5 y 6 (+ proyecto de referencia smartlibrary.zip)
Explicación de las decisiones de diseño (máx. 1 página)	Sección final
Preguntas de revisión y defensa	Actividad 7 (incluye matriz de trazabilidad)

Reglas del caso que se rastrean en todo el documento: (R1) solo un libro disponible puede prestarse; (R2) el préstamo registra estudiante, libro y fecha; (R3) al prestarlo, el libro deja de estar disponible; (R4) una solicitud inválida no crea préstamo; (R5) un libro puede tener varios préstamos históricos, pero solo uno activo.

Actividad 1 · Leer y cuestionar el modelo

1. ¿Qué datos y operaciones son propios del dominio?

Son del dominio los datos y comportamientos que existirían aunque no hubiera web ni base de datos, es decir, los que describen la biblioteca:

Clase	Datos del dominio	Operaciones del dominio
Estudiante	id, nombre, correo	Constructor y consultas (getId, getNombre, getCorreo).
Libro	id, titulo, disponible	isDisponible() y marcarPrestado(): son comportamiento real del negocio (R1, R3). Libro es el experto en su disponibilidad.
Prestamo	id, fechaPrestamo, fechaDevolucion [0..1], estudiante, libro	Constructor que exige estudiante, libro y fecha (R2). Se propone estaActivo() = fechaDevolucion es nula (R5).
GestorPrestamos	— (no es del dominio)	Sus listas y get/set son almacenamiento (rol de repositorio). registrarPrestamo() es coordinación del caso de uso (rol de servicio), no un concepto de la biblioteca.

2. ¿Por qué Libro–Prestamo y Estudiante–Prestamo son asociaciones y no composiciones?

Una composición exige una relación todo-parte con **propiedad exclusiva** y **ciclo de vida dependiente**: si se destruye el todo, se destruyen las partes, y cada parte pertenece a un solo todo a la vez. Nada de eso ocurre aquí:

- El libro y el estudiante existen antes del préstamo y siguen existiendo después de que se devuelve o se elimina un préstamo. Sus ciclos de vida son independientes.
- Un mismo libro participa en 0..* préstamos a lo largo del tiempo (historial) y un estudiante también; no son "parte" de un único préstamo.
- Ni el libro "contiene" sus préstamos ni el préstamo "contiene" al libro: el préstamo es un hecho que **vincula** a dos objetos independientes. Eso es exactamente una asociación (referencia estructural persistente) con multiplicidad 0..* — 1.

Tampoco se justifica la agregación: no hay semántica todo-parte, y en UML la agregación no agrega significado preciso a una asociación normal.

3. ¿Qué operaciones son necesarias para consultar, validar y guardar?

Propósito	Operaciones necesarias
Consultar	Buscar estudiante por id; buscar libro por id; listar préstamos (GET /api/prestamos).
Validar	Que la petición traiga estudianteId y libroId (formato); que el estudiante exista; que el libro exista; que el libro esté disponible (R1/R5).
Guardar	Marcar el libro como prestado (R3); crear el préstamo con estudiante, libro y fecha (R2); persistir libro y préstamo en una sola transacción, de modo que si algo falla no quede nada a medias (R4).

4. ¿Qué responsabilidades faltan para recibir una petición HTTP? ¿Y para acceder a datos?

Para HTTP (capa de presentación): asociar la ruta y el método (POST /api/prestamos); convertir el JSON de entrada a un objeto (PrestamoRequest); validar el formato básico; invocar el caso de uso; convertir el resultado a JSON (PrestamoResponse); elegir el código de estado (201, 200) y la cabecera Location; traducir errores a 404, 409 o 400. Ninguna clase del modelo inicial hace esto.

Para acceso a datos (capa de persistencia): buscar por identificador, guardar y actualizar, listar, generar los identificadores, mapear objetos a tablas y participar en transacciones. En el modelo inicial esto lo "simulan" las listas en memoria de GestorPrestamos, que no persisten nada.

Observaciones sobre el diagrama inicial

La Figura 1 (página siguiente) es el modelo inicial con siete observaciones (O1–O7) ancladas a las clases. En resumen:

- **O1** GestorPrestamos tiene baja cohesión: almacena colecciones y coordina el caso de uso.
- **O2** registrarPrestamo recibe objetos, pero HTTP envía identificadores: falta buscar por id y responder 404.
- **O3** No existe ninguna clase que reciba HTTP ni que produzca 201/404/409.
- **O4** Libro es experto en su disponibilidad, pero setDisponible() público permite saltarse R1/R3.
- **O5** Prestamo es quien guarda las referencias: la navegabilidad real es Prestamo → Estudiante y Prestamo → Libro.
- **O6** Constructores con id y setters de estudiante/libro permiten objetos inconsistentes; el id debe generarlo la persistencia.
- **O7** Las relaciones con Prestamo son asociaciones, no composiciones (ciclos de vida independientes).

Figura 1 · Diagrama inicial anotado

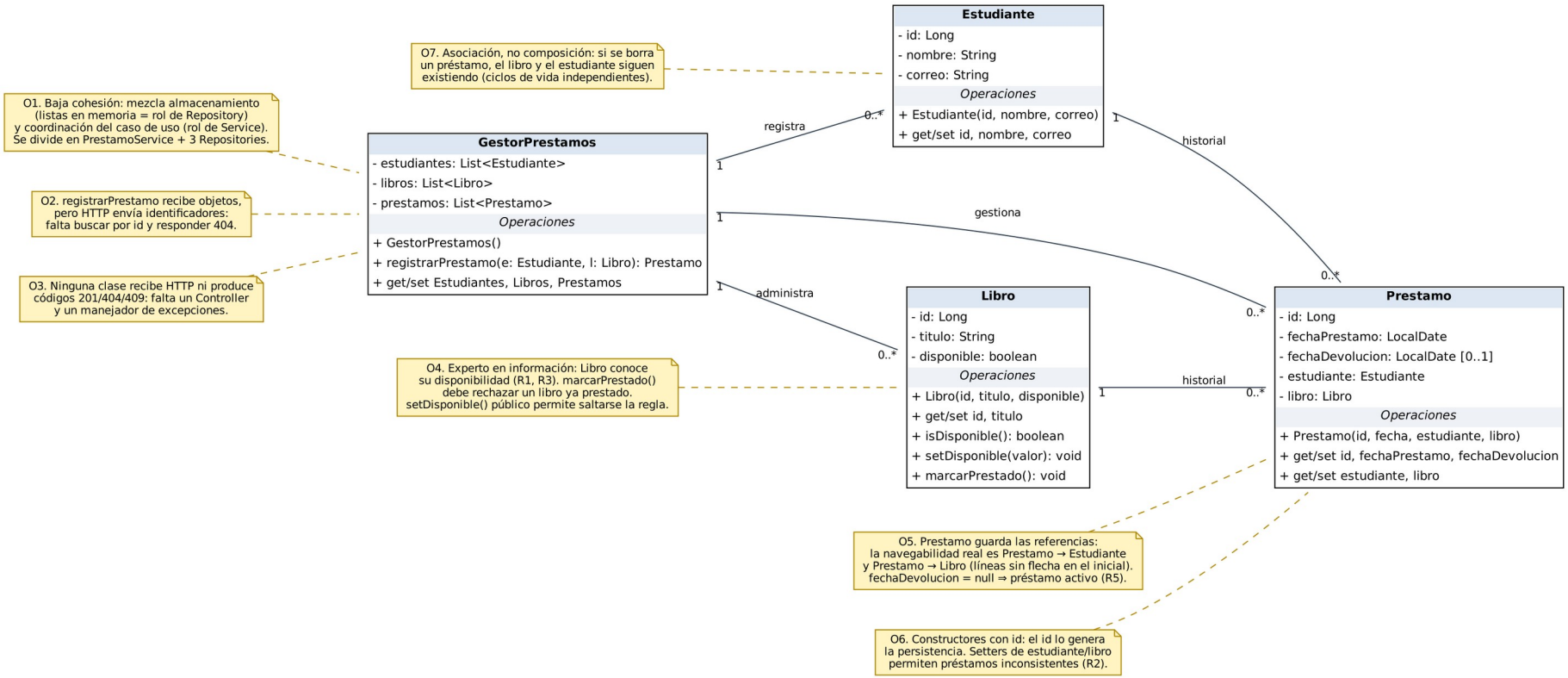


Figura 1. Modelo inicial de SmartLibrary con observaciones O1–O7 (notas amarillas).

Actividad 2 · Asignar responsabilidades

Las entidades Estudiante, Libro y Prestamo se mantienen como modelo de dominio usado por las capas; no son una cuarta capa. Se usa "Controller → Service → Repository".

Operación o decisión	Clase responsable	Justificación
Recibir POST /api/prestamos	PrestamoController.registrar(request: PrestamoRequest)	Es la única clase que conoce HTTP: ruta, método, JSON y @Valid. Solo traduce la petición y delega; no contiene reglas de negocio (alta cohesión).
Buscar estudiante por identificador	EstudianteRepository.findById(id), invocado por PrestamoService	El acceso a datos se encapsula en el repositorio (Fabricación pura / Indirección). El servicio decide qué pasa si no existe: lanza RecursoNoEncontradoException → 404.
Buscar libro por identificador	LibroRepository.findById(id), invocado por PrestamoService	Igual que el anterior; cada repositorio gestiona una sola entidad (cohesión). Si no existe → 404.
Validar disponibilidad	PrestamoService, consultando Libro.isDisponible()	Libro es el Experto en información (conoce su estado); el servicio aplica la regla del caso de uso y decide la consecuencia (LibroNoDisponibleException → 409). Así la regla R1 no depende de HTTP ni de la BD.
Crear el préstamo	PrestamoService: new Prestamo(estudiante, libro, LocalDate.now())	Creador GRASP: el servicio tiene los datos para inicializarlo (estudiante y libro ya validados). El constructor exige los tres datos (R2) y no recibe id.
Cambiar disponibilidad	Libro.marcarPrestado(), invocado por PrestamoService	Experto en información y encapsulamiento: el libro cambia su propio estado y se protege (rechaza prestarse si no está disponible). Se elimina setDisponible() público (R3).
Guardar cambios	LibroRepository.save(libro) y PrestamoRepository.save(prestamo), dentro de la transacción de PrestamoService	Los repositorios persisten; el servicio delimita la transacción (@Transactional) para que cambio de disponibilidad y creación del préstamo sean "todo o nada" (R3 + R4).
Devolver respuesta HTTP	PrestamoController (201 + Location + PrestamoResponse) y GlobalExceptionHandler (404, 409, 400)	Presentación traduce resultados y excepciones a códigos HTTP. El servicio no conoce HTTP, lo que reduce el acoplamiento y permite reutilizarlo desde pruebas u otras interfaces.

Actividad 3 · Modelo de clases por capas

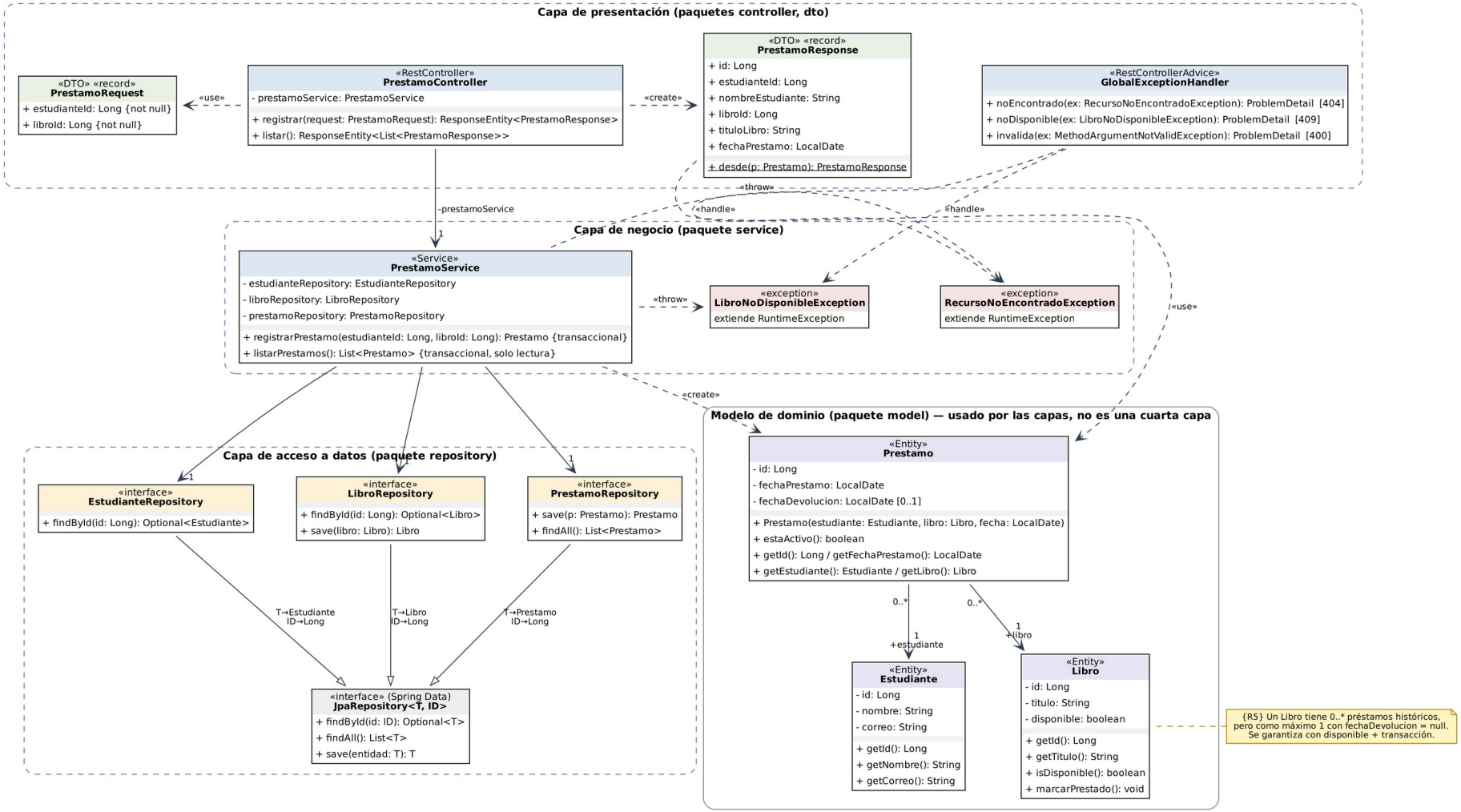


Figura 2. Diagrama de clases por capas. Línea continua con flecha abierta = asociación dirigida; discontinua = dependencia; triángulo hueco = generalización.

Justificación de relaciones y multiplicidades

Criterio aplicado (según el taller): **asociación** cuando una clase conserva una referencia (atributo) a otra; **dependencia** cuando solo la usa puntualmente (parámetro, variable local, retorno, excepción).

Relación	Tipo UML	Multiplicidad	Justificación
PrestamoController → PrestamoService	Asociación dirigida	1	El controlador guarda el servicio en un atributo final inyectado por constructor. Unidireccional: el servicio no conoce al controlador.
PrestamoService → EstudianteRepository / LibroRepository / PrestamoRepository	Asociación dirigida (×3)	1 cada una	El servicio mantiene un atributo por repositorio. Respeta la dirección Controller → Service → Repository; el controlador no toca repositorios.
PrestamoController ⇢ PrestamoRequest	Dependencia «use»	—	Solo lo recibe como parámetro del método registrar().
PrestamoController ⇢ PrestamoResponse	Dependencia «create»	—	Lo crea al responder; no lo guarda.
PrestamoService ⇢ Prestamo	Dependencia «create»	—	Crea el préstamo dentro del método; no conserva referencia.
PrestamoService ⇢ excepciones	Dependencia «throw»	—	Lanza RecursoNoEncontradoException y LibroNoDisponibleException.
GlobalExceptionHandler ⇢ excepciones	Dependencia «handle»	—	Las recibe como parámetro y las traduce a 404/409.
XRepository —▷ JpaRepository<T, ID>	Generalización entre interfaces	—	Una interfaz extiende a otra con ligadura de parámetros (T→entidad, ID→Long). No es realización.
Prestamo → Estudiante	Asociación dirigida	Prestamo 0..* — 1 Estudiante	Cada préstamo tiene exactamente un estudiante; un estudiante puede tener cero o muchos préstamos. Se conserva del modelo inicial.
Prestamo → Libro	Asociación dirigida + restricción {R5}	Prestamo 0..* — 1 Libro	Historial de muchos préstamos, pero como máximo uno activo (fechaDevolucion nula).

¿Dónde hay realización de interfaces? En nuestro código, en ninguna parte, y es intencional: los repositorios son interfaces cuya implementación la genera Spring Data en tiempo de ejecución, así que dibujar una clase "RepositoryImpl" sería modelar infraestructura que no escribimos. Tampoco se crea una interfaz PrestamoService con una clase PrestamoServiceImpl porque hay una sola implementación (principio YAGNI); si en el futuro hubiera varias estrategias de préstamo, entonces sí aparecería una realización.

Navegabilidad del dominio. Libro y Estudiante no guardan una lista de préstamos: el historial se consulta mediante PrestamoRepository. Así se evitan referencias bidireccionales, ciclos al convertir a JSON y acoplamiento innecesario.

Otras decisiones visibles en el diagrama: se usan DTO (PrestamoRequest/PrestamoResponse) para no exponer las entidades en el contrato de la API; se agrega GlobalExceptionHandler porque los códigos 404 y 409 requieren manejo explícito de excepciones; GestorPrestamos desaparece y sus responsabilidades se reparten entre PrestamoService y los tres repositorios.

Actividad 4 · Diagrama de secuencia — POST /api/prestamos

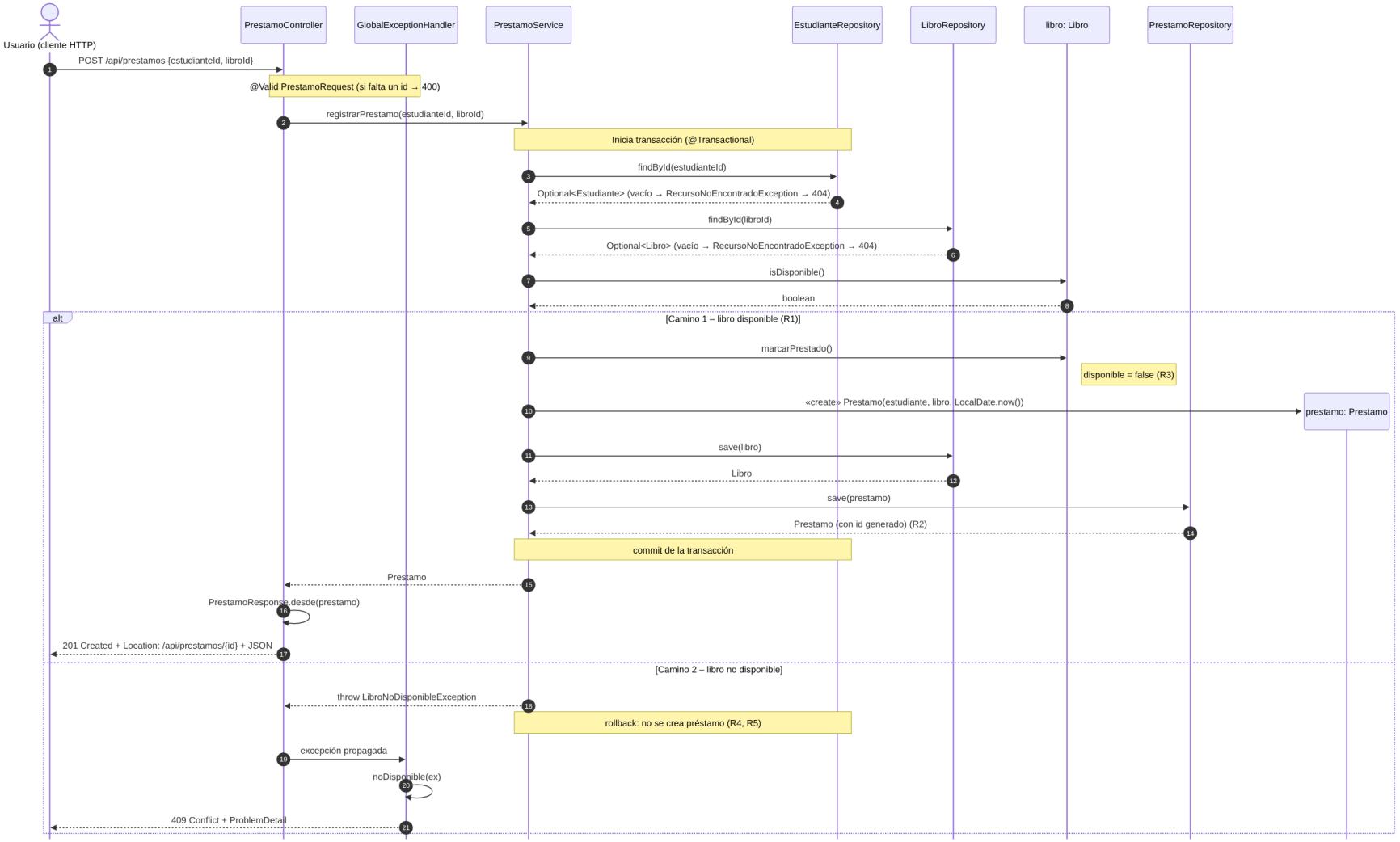


Figura 3. Secuencia de «Registrar préstamo» con dos caminos (fragmento alt): préstamo exitoso y libro no disponible.

Los caminos de **estudiante o libro inexistente** se indican en los mensajes 4 y 6: si findById devuelve un Optional vacío, el servicio lanza RecursoNoEncontradoException y GlobalExceptionHandler responde 404, de forma análoga al camino 2.

Verificación: cada mensaje corresponde a una operación del diagrama de clases

Mensaje(s)	Operación en el diagrama de clases
1	PrestamoController.registrar(request: PrestamoRequest)
2, 15	PrestamoService.registrarPrestamo(estudianteId, libroId): Prestamo
3–4	EstudianteRepository.findById(id): Optional<Estudiante>
5–6	LibroRepository.findById(id): Optional<Libro>
7–8	Libro.isDisponible(): boolean
9	Libro.marcarPrestado(): void
10	Prestamo(estudiante, libro, fecha) — constructor; dependencia «create» Service ⇢ Prestamo
11–12	LibroRepository.save(libro): Libro
13–14	PrestamoRepository.save(p): Prestamo
16	PrestamoResponse.desde(p: Prestamo) — operación estática
17	Retorno ResponseEntity<PrestamoResponse> de registrar() con 201 Created
18	Dependencia «throw» Service ⇢ LibroNoDisponibleException
19–21	GlobalExceptionHandler.noDisponible(ex): ProblemDetail → 409 Conflict

Actividad 5 · Preparación de la implementación

Configuración del proyecto en Spring Initializr (start.spring.io o el IDE):

Parámetro	Valor
Project / Language	Maven / Java
Spring Boot	Versión estable 3.5.x (o la que indique el profesor)
Group / Artifact	co.edu / smartlibrary
Packaging / Java	Jar / 21
Dependencias	Spring Web, Spring Data JPA, Validation, H2 Database

Organización de paquetes (las clases de cada paquete coinciden con la Figura 2):

```
co.edu.smartlibrary
├── controller   PrestamoController
├── service      PrestamoService
├── repository   EstudianteRepository, LibroRepository, PrestamoRepository
├── model        Estudiante, Libro, Prestamo
├── dto          PrestamoRequest, PrestamoResponse
└── exception    RecursoNoEncontradoException, LibroNoDisponibleException,
                 GlobalExceptionHandler
```

El paquete **exception** es una adición sugerida; si el profesor prefiere solo los cinco paquetes del enunciado, las excepciones pueden ir en service y el manejador en controller sin cambiar el diseño.

Configuración mínima de H2 (application.properties) y datos iniciales de prueba (data.sql):

```
spring.datasource.url=jdbc:h2:mem:smartlibrary
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.defer-datasource-initialization=true
```

Datos iniciales	Valores	Para qué escenario
Estudiantes	1 Ana Gómez · 2 Luis Pérez	Préstamos válidos
Libros	1 Diseño de Software (disponible) · 2 Patrones GRASP (disponible) · 3 Clean Architecture (NO disponible)	Éxito (libro 1) y conflicto (libro 3)
Préstamos	Un préstamo activo: estudiante 2 – libro 3	Coherencia con R5 y consulta GET
Ids inexistentes	estudianteId = 99, libroId = 99	Escenarios 404

Criterios de verificación antes de programar: (1) el controlador no importa ningún repositorio; (2) el servicio no importa clases de HTTP; (3) Libro no expone setDisponible(); (4) registrarPrestamo es transaccional; (5) cada operación de la Figura 2 aparece en el código con el mismo nombre.

Actividad 6 · Implementación guiada y pruebas

Escenarios de verificación. Las peticiones pueden ejecutarse con Postman, Thunder Client, el archivo pruebas.http del proyecto de referencia o curl, por ejemplo:

```
curl -i -X POST http://localhost:8080/api/prestamos \
     -H "Content-Type: application/json" \
     -d '{"estudianteId":1,"libroId":1}'
```

#	Escenario	Petición	Resultado esperado	Regla	Evidencia (captura / resultado obtenido)
1	Préstamo de libro disponible	POST {"estudianteId":1,"libroId":1}	201 Created, cabecera Location y préstamo registrado; libro 1 queda no disponible	R1, R2, R3	
2	Libro ya prestado	POST {"estudianteId":1,"libroId":3}	409 Conflict; sin nuevo préstamo	R1, R4, R5	
3	Estudiante inexistente	POST {"estudianteId":99,"libroId":2}	404 Not Found	R4	
4	Libro inexistente	POST {"estudianteId":1,"libroId":99}	404 Not Found	R4	
5	Consulta de préstamos	GET /api/prestamos	200 OK con los registros	R2	
6 (extra)	Petición sin estudianteId	POST {"libroId":2}	400 Bad Request	R4	

La última columna se completa con la captura de pantalla o el código de estado que obtengan en el laboratorio. El proyecto de referencia incluye además la prueba automatizada **PrestamoControllerIT** (MockMvc), con un método por escenario; ejecutar `mvn test` y adjuntar el resultado sirve como evidencia adicional. Un buen control extra: después del escenario 2, hacer GET y comprobar que el número de préstamos no cambió (R4).

Decisiones de la implementación que deben reflejarse en UML

- Los repositorios son **interfaces** que extienden JpaRepository<T, Long> → generalización entre interfaces, no clases.
- La inyección por constructor crea atributos final → **asociaciones dirigidas** Controller → Service → Repository.
- @ManyToOne en Prestamo hacia Estudiante y Libro → asociación dirigida 0..* — 1 desde Prestamo.
- @GeneratedValue en Prestamo.id → el constructor de Prestamo ya no recibe id.
- @Transactional en registrarPrestamo → propiedad {transaccional} en la operación del servicio.

- @RestControllerAdvice → clase GlobalExceptionHandler y las dos excepciones con sus dependencias.
- Records PrestamoRequest/PrestamoResponse con @NotNull → clases «DTO» con restricción {not null}.

Transacción y concurrencia

Marcar el libro como prestado y guardar el préstamo deben ocurrir juntos: si guardar el préstamo falla, la transacción hace rollback y el libro vuelve a estar disponible. Una consideración adicional es la concurrencia: dos peticiones simultáneas podrían leer disponible = true al mismo tiempo. Se mitiga con bloqueo optimista (@Version en Libro) o con una restricción en la base de datos que impida dos préstamos activos del mismo libro; queda fuera del alcance del taller.

Actividad 7 · Revisión y defensa

1. ¿Qué responsabilidades cambiaron del modelo inicial al modelo por capas?

- **GestorPrestamos desapareció.** Su almacenamiento en listas pasó a los tres repositorios (persistencia real en H2) y su método registrarPrestamo pasó a PrestamoService, que ahora recibe identificadores en lugar de objetos.
- **Aparecieron responsabilidades nuevas:** recibir HTTP y responder (PrestamoController), traducir errores a 404/409/400 (GlobalExceptionHandler) y definir el contrato de entrada/salida (DTO).
- **Libro conserva** su disponibilidad y marcarPrestado(), pero ahora la protege (sin setDisponible público). **Prestamo** ya no recibe id ni tiene setters: se crea completo y la base de datos genera el id.

2. ¿Qué patrón GRASP se reconoce en la coordinación del caso de uso? ¿Por qué un controlador GRASP no equivale necesariamente a @RestController?

Se reconoce el patrón **Controlador** (controlador de caso de uso): PrestamoService es el primer objeto, más allá de la capa de interfaz, que recibe la operación del sistema "registrar préstamo" y coordina a los demás objetos sin hacer él mismo todo el trabajo. También aparecen **Experto en información** (Libro decide sobre su disponibilidad), **Creador** (el servicio crea Prestamo porque tiene los datos de inicialización), **Fabricación pura** e **Indirección** (repositorios y servicio), y **Bajo acoplamiento / Alta cohesión**.

Un controlador GRASP es un concepto de diseño **independiente de la tecnología**: es quien recibe y coordina un evento del sistema. @RestController es una anotación de Spring para una clase de la **capa de presentación** que adapta HTTP (rutas, JSON, códigos de estado). Según Larman, la capa de interfaz no debería contener lógica de la aplicación, sino delegar en el controlador. Por eso PrestamoService puede ser invocado igual desde una prueba, una consola o una cola de mensajes, sin cambiar, mientras que el @RestController es solo una de esas posibles puertas de entrada.

3. ¿Dónde se ubica la regla de disponibilidad y por qué?

En la **capa de negocio**: PrestamoService aplica la regla del caso de uso (si el libro no está disponible, lanza LibroNoDisponibleException) y Libro, como experto en información, protege su propio estado en marcarPrestado(). No va en el controlador porque se duplicaría en cada punto de entrada y mezclaría HTTP con negocio; no va en el repositorio porque su responsabilidad es solo persistir. Además, la regla debe ejecutarse dentro de la transacción que crea el préstamo, y esa transacción la delimita el servicio.

4. ¿Qué relaciones UML conservaron y cuáles modificaron?

Relación	Decisión
Estudiante 1 — 0..* Prestamo	Se conserva la asociación y su multiplicidad; se hace explícita la navegabilidad Prestamo → Estudiante.
Libro 1 — 0..* Prestamo	Se conserva y se agrega la restricción {R5: a lo sumo un préstamo activo}.
GestorPrestamos 1 — 0..* Estudiante / Libro / Prestamo	Se eliminan junto con la clase. Se reemplazan por asociaciones Service → Repository (multiplicidad 1); los repositorios consultan la BD, no guardan listas.
Nuevas	Asociación Controller → Service; dependencias hacia DTO y excepciones; generalización de repositorios hacia JpaRepository.

Relación	Decisión
Agregación / composición	Siguen sin usarse: no hay relaciones todo-parte con ciclo de vida dependiente.

5. ¿Cómo se comprueba la trazabilidad requisito → clase → operación → prueba?

Con una matriz que conecta cada regla con la clase y la operación que la implementan y con la prueba que la verifica. Si una fila queda vacía, falta diseño o falta prueba:

Requisito	Clase	Operación	Prueba (escenario / método)
R1 Solo se presta un libro disponible	PrestamoService, Libro	registrarPrestamo(), isDisponible()	Esc. 2 → 409 · libroYaPrestado_devuelve409_sinNuevoPrestamo
R2 Registra estudiante, libro y fecha	Prestamo, PrestamoRepository	Prestamo(estudiante, libro, fecha), save()	Esc. 1 y 5 → cuerpo con estudianteId, libroId y fechaPrestamo
R3 El libro deja de estar disponible	Libro	marcarPrestado()	Esc. 1: segundo préstamo del mismo libro → 409
R4 Solicitud inválida no crea préstamo	PrestamoService (@Transactional), GlobalExceptionHandler, PrestamoRequest	registrarPrestamo(), noEncontrado(), invalida()	Esc. 2, 3, 4 y 6 + GET que confirma que no hay registro nuevo
R5 Solo un préstamo activo por libro	Libro, Prestamo	isDisponible(), estaActivo()	Esc. 2 (el libro 3 ya tiene un préstamo activo)
Consulta de préstamos	PrestamoController, PrestamoService	listar(), listarPrestamos()	Esc. 5 → 200 · consultaDePrestamos_devuelve200

Explicación de las decisiones de diseño

El modelo inicial concentraba en GestorPrestamos dos responsabilidades muy distintas, guardar colecciones y coordinar el préstamo, y no tenía a nadie que atendiera HTTP ni que persistiera datos. Por eso lo dividimos en tres capas con una sola dirección de dependencias: **PrestamoController → PrestamoService → Repositories**. Cada capa tiene un motivo de cambio distinto: el contrato HTTP, las reglas del caso de uso o la tecnología de persistencia.

Presentación. PrestamoController solo traduce HTTP: recibe un PrestamoRequest validado, delega en el servicio y devuelve 201 con la ubicación del recurso. Los errores no se manejan con if en el controlador, sino con excepciones de negocio que GlobalExceptionHandler convierte en 404, 409 o 400. Usamos DTO para que el contrato de la API no dependa de las entidades JPA.

Negocio. PrestamoService es el controlador GRASP del caso de uso: busca estudiante y libro, verifica la disponibilidad, pide al libro que se marque como prestado, crea el préstamo y guarda todo dentro de una transacción. La regla de disponibilidad vive aquí y en Libro (experto en información), no en el controlador ni en el repositorio, para que se cumpla sin importar quién invoque el caso de uso. La transacción asegura que R3 y R4 se cumplan juntos: o se presta el libro y se registra el préstamo, o no ocurre nada.

Datos. Los tres repositorios son interfaces que extienden JpaRepository; por eso en UML aparecen como generalizaciones entre interfaces y no como realizaciones, ya que la implementación la genera Spring Data. Mantenemos un repositorio por entidad para conservar la cohesión.

Dominio y relaciones. Conservamos las asociaciones Prestamo 0..* — 1 Estudiante y Prestamo 0..* — 1 Libro, ahora navegables solo desde Prestamo, que es quien guarda las referencias. No usamos composición ni agregación porque libros y estudiantes existen con independencia de sus préstamos. Distinguimos asociaciones (atributos inyectados que se conservan) de dependencias (uso puntual de DTO, excepciones y la creación del préstamo). R5 se expresa como restricción sobre la asociación Libro–Prestamo y se garantiza con el atributo disponible y la transacción.

Resultado. Hay alta cohesión (cada clase tiene un propósito), bajo acoplamiento (el controlador no conoce repositorios y el servicio no conoce HTTP) y trazabilidad completa: cada mensaje del diagrama de secuencia es una operación del diagrama de clases y cada regla R1–R5 tiene una prueba que la verifica.